Reporte

| | |
|---|---|
| **Nombre**:<br>1. Roberto Santana Trujillo<br>2. Isaías Leonardo Salazar Torres | **Matrícula**:<br>2227084<br>2227016 |
| **Nombre del curso:** Componentes de lógica de negocio | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Modulo Netacad**: 3 | **Actividad**: U3 – Diseño Arquitectural del Sistema - FerreMax |
| **Fecha**: 17/10/2026 | |
| **Bibliografia (Formato APA)**<br>Presentación mostrada en clase. | |

## Objetivo:

Continuando con el caso de negocio simulado de la actividad anterior, en nuestro caso la ferretería FerreMax, se expandirá sobre lo construido en la unidad anterior, expandiendo los requerimientos y la lógica del negocio a un diseño arquitectural del sistema a realizar como proyecto final. En este resumen se incluye el patrón de lógica de negocio a usar, el mapa de dominio, service layer, diagramas de clases, diagramas de secuencias y ADR's para este diseño.

Link del repositorio en Github:

**https://github.com/Santana-R/ferremax-u3**

# Procedimiento

## Etapa 1: Preparación del caso

Antes de detallar los casos de uso, es importante volver a considerar la base del negocio FerreMax, y lo realizado en la unidad 2.

La empresa FerreMax opera tres sucursales y un almacén central dedicados a la venta de herramientas y materiales de construcción. El análisis previo evidenció serias inconsistencias, como el riesgo de confirmar ventas contra mercancía ya comprometida, el uso de cotizaciones vencidas (gestionadas en hojas de cálculo) y una visibilidad limitada de inventario, lo que derivaba en promesas de mercancía sin sustento.

FerreMax requiere coordinar cotizaciones, pedidos e inventario sin comprometer mercancía inexistente. La arquitectura propuesta organiza las reglas del negocio con Domain Model, una Service Layer y adaptadores de persistencia. El recorrido detallado cubre la consulta de disponibilidad, la revalidación de cotizaciones, la conversión a pedido, la confirmación con reserva y la cancelación antes del surtido.

Continuidad de la unidad 2:

| Fuente | Uso en U3 |
|---|---|
| EQ17FerreMax_BRD.pdf | Contexto de tres sucursales y almacén central; BN01-BN10, BR01-BR16, RNF01-RNF08, alcance y supuestos. |
| EQ17FerreMax_UserStories.pdf | US01-US20 y sus Gherkin; casos normales y rechazos; reserva atómica y competencia por última unidad. |
| EQ17FerreMax_Glosario.pdf | Lenguaje: SKU, física, comprometida, disponible, tránsito, cotización, pedido, reserva, crédito, transferencia y entrega. |
| EQ17FerreMax_BPMN.pdf | BPMN cotejado visualmente. Se registra H07: la reserva aparece antes del procesamiento del pago o la validación del crédito. |

Por último, se encontraron unos hallazgos de continuidad en estas fuentes anteriores, es importante registrarlas:

**HC1.** La User Story US01 usa el criterio de aceptación AC-US01-01 para dos escenarios distintos. La matriz U3 (más adelante en este documento) propone identificar el escenario «SKU inexistente» como el criterio de aceptación AC-US01-02, conservando la referencia al ID original repetido.

**HC2**. El objetivo de negocio OB01 plantea reducir 80% las ventas sin mercancía; mientras que el criterio de éxito CE01 exige eliminarlas. Se propone conservar OB01 como meta de mejora frente a la línea base y tratar CE01 como aspiración; la arquitectura protege el invariante de no reservar mercancía inexistente.

**HC3.** No se especifica qué resultado financiero corresponde cuando la caja registra un pago autorizado y después la reserva falla. El sistema revierte pedido y reserva; la atención del pago ya autorizado queda pendiente de política comercial. No se simula una devolución automática.

**HC4.** El BPMN dibuja "Reservar para pedido confirmado" antes de "Procesar pago" o "Validar crédito" y del gateway "¿Autorizado?". Esta unidad 3 sigue la regla de negocio BR05 y las user stories US06/US09: autorización registrada, confirmación y reserva atómicas.

**Alcance:**

El proceso detallado va desde la disponibilidad y la cotización hasta la confirmación del pedido y la liberación de su reserva. Todos los casos aparecen en User Stories de la unidad 2. Los demás procesos se representan para conservar el panorama de FerreMax y sus vínculos.

La confirmación ejecutable es de contado y usa una ubicación por pedido. Esto delimita el prototipo, no cambia las reglas del negocio sobre surtido multiorigen, crédito o parciales. Las consultas sí devuelven las cuatro ubicaciones de demostración. Los datos, referencias de pago y actores del demo son simulados.

La vigencia de siete días proviene de un acuerdo simulado del BRD y se pasa como parámetro.

Para continuar con el diseño arquitectural, se tomaron algunas User Stories de la unidad 2 para tomar cinco casos de uso:

**Casos de uso:**

| ID | Caso | Actor | Reglas relacionadas (U2) | Resultado |
|---|---|---|---|---|
| **CU01** | Consultar disponibilidad global | Vendedor | BN01; BR04; US01/US02 | DTO por ubicación con física, comprometida, tránsito y disponible; SKU inexistente sin existencias inventadas. |
| **CU02** | Revalidar cotización vencida | Vendedor | BN03; BR01-03; US04 | Nueva versión vigente; versión anterior conservada; no crea reserva. |
| **CU03** | Convertir cotización aceptada en pedido | Vendedor | BN03/BN04; BR01/BR03; US05 | Pedido pendiente ligado al folio y versión; disponibilidad consultada nuevamente. Vencida: rechazo y solicitud de revalidación. |
| **CU04** | Confirmar pedido de contado y reservar | Caja | BN02; BR04/BR05; RNF02; US06/US09 | Pedido confirmado y reserva como una operación. Pago rechazado o stock insuficiente: no confirma y no reserva. |
| **CU05** | Cancelar pedido antes del surtido | Vendedor con permiso de cancelación | BN02; BR06/BR16; RNF03; US10 | Pedido cancelado, reserva liberada y modificación auditada. Pedido surtido: rechazo. |

## Etapa 2: Elegir el patrón de lógica de negocio

Después de resumir lo obtenido en la unidad 2 y documentar los casos de uso para el diseño arquitectural de este sistema, es momento de elegir el patrón de lógica de negocio a utilizar. Se compararán tres patrones: Transaction Script, Table Module y Domain Model en una tabla comparativa, para luego elegir el patrón que más concuerde con el negocio.

| Criterio | Transaction Script | Table Module | Domain Model | Evidencia U2 |
|---|---|---|---|---|
| Cotizaciones con vigencia y versiones | Validaciones por procedimiento; riesgo de repetirlas en convertir y revalidar. | Organiza versiones como filas; invariantes por conjunto pueden quedar dispersas. | Cotizacion protege vigencia y produce otra versión sin borrar la anterior. | BR01-03; US03-05 |
| Confirmación, inventario y concurrencia | Una transacción puede resolverlo, pero la coordinación y reglas tienden a mezclarse. | Procesamiento por tablas adecuado para lotes; debe reconstruir vínculos entre pedido y reservas. | Pedido e InventarioUbicacion protegen sus invariantes; UoW coordina su guardado. | BR04-06; RNF02; AC-US09-01/02 |
| Estado y ciclo de vida | Cambios de estado repartidos entre scripts; exige disciplina para evitar caminos contradictorios. | Estados expresados en filas; comportamiento separado de conceptos del negocio. | Métodos confirmar y cancelar expresan transiciones permitidas y rechazos. | US06/US10/US15; BR05/BR06 |
| Variantes comerciales | Cada variante añade condiciones a los procedimientos existentes. | Cambia módulos centrados en datos; puede ser útil si predominan operaciones por conjunto. | Políticas de crédito y devolución pueden evolucionar con contratos propios. | BR09-11/BR15; BN05/BN09 |
| Costo para el equipo de cómputo | Menor estructura inicial; apropiado si el caso se redujera a operaciones simples. | Costo intermedio; natural en aplicaciones organizadas alrededor de datasets. | Más clases, interfaces y revisión de límites; exige explicar responsabilidades y evitar clases sin comportamiento. | 16 reglas BR y excepciones documentadas |

| Criterio | Transaction Script | Table Module | Domain Model | Evidencia U2 |
|---|---|---|---|---|
| Consultas operativas | Consulta directa por procedimiento. | Buena afinidad con agregación y reportes por tablas. | Modelo de lectura independiente: no obliga a reconstruir agregados para cada consulta. | BN01/BN10; RNF01 |

**Decisión de patrón de lógica de negocio:**

Transaction Script puede implementar correctamente las operaciones seleccionadas. Su ventaja es la estructura inicial sencilla: consultar, verificar condiciones, actualizar y devolver un resultado. También puede usar una transacción para evitar cambios parciales. Sin embargo, se descarta como patrón principal por su costo de mantenimiento en este caso, no porque sea incapaz de proteger el inventario.

En FerreMax, la regla «una cotización no reserva» aparece en la emisión, la revalidación y la conversión; la disponibilidad se consulta y se comprueba otra vez al confirmar; la cancelación afecta pedido y reservas. Si cada procedimiento conserva sus propias validaciones, una corrección debe repetirse en varios lugares. Puede mantenerse coherente, pero requiere controles continuos para evitar divergencia. El riesgo crece con crédito excepcional, transferencias y parciales.

En el caso de Table Module, este concentra comportamiento sobre conjuntos de registros y resulta razonable cuando la aplicación gira alrededor de datasets. Podría facilitar reportes operativos y procesamiento por ubicación. Su costo en FerreMax sería expresar el comportamiento de cotizaciones, pedidos y reservas mediante relaciones entre filas, mientras su ciclo de vida y reglas se distribuyen en módulos centrados en tablas.

La consulta global necesita una proyección eficiente, pero eso no obliga a usar Table Module como patrón de negocio. Se propone una consulta SQL de solo lectura para el DTO de disponibilidad. La separación entre un modelo de lectura y los agregados de escritura resuelve esa necesidad sin adoptar un segundo patrón principal artificial.

Entonces, la última opción es la más aceptable. Domain Model permite ubicar la vigencia en Cotizacion, las transiciones en Pedido y el compromiso de cantidades en InventarioUbicacion. Las operaciones públicas expresan intenciones del negocio: revalidar, convertir, confirmar, reservar, liberar y cancelar. La identidad y el estado se conservan, y el servicio de aplicación coordina objetos que ya conocen sus reglas.

La decisión se apoya especialmente en BR01-03, BR04-06 y RNF02/AC-US09-02. Estas evidencias exigen conservar versiones, distinguir compromiso de compra de compromiso de inventario y proteger una última unidad frente a dos pedidos. La claridad de responsabilidades importa más que la cantidad de clases.

El costo aceptado es una estructura mayor: objetos, contratos, adaptadores y pruebas de colaboración. Se controla concentrando el desarrollo en cinco casos y evitando capas que no resuelvan una necesidad. Domain Model no sustituye las transacciones ni los bloqueos del motor de SQL: las reglas se verifican sobre datos recuperados de manera consistente.

Por tanto, el patrón de lógica de negocio elegido es Domain Model.

## Etapa 3: Construir el mapa de dominio

Para construir el mapa de dominio, es importante retomar el glosario de dominio realizado en la unidad 2 y actualizarlo con nuevos términos y correcciones para el nuevo patrón de lógica de negocio escogido:

Glosario actualizado (Lenguaje ubicuo):

| Término | Definición |
|---|---|
| Cotización | Aggregate Root. Protege las versiones y vigencia. |
| Pedido | Aggregate Root. Ciclo de vida y transición de la compra. |
| PartidaPedido | Renglón de una cotización, pedido, transferencia o devolución que identifica producto, cantidad y datos aplicables. |
| Reserva | Entidad. Cantidad comprometida para un pedido confirmado. |
| Ubicación | Sucursal o almacén central donde se controla inventario. |

| | |
|---|---|
| InventarioUbicacion | Aggregate Root. Estado del inventario por SKU y ubicación. |
| CuentaCrédito | Registro de límite, saldo, disponible, bloqueo y condiciones de pago de un cliente empresarial. |
| ClienteEmpresarial | Organización registrada que puede tener condiciones comerciales y cuenta de crédito. |
| Dinero / Vigencia | Value Objects inmutables usados en el dominio. |
| Entrega | Registro de receptor, fecha/hora y soporte que demuestra el resultado de la entrega. |
| Devolución | Solicitud para retornar un producto vendido, evaluada contra comprobante, condición y política. |
| Evento de dominio | Hecho ocurrido y confirmado; por ejemplo PedidoConfirmado. No es una orden para confirmar. |
| Application Service | Coordina un caso de uso y solicita acciones al dominio; no decide todas las reglas. |
| Repository | Contrato para recuperar o guardar objetos del negocio; su implementación contiene SQL. |
| Unit of Work | Límite que confirma juntos los cambios de la operación o los revierte ante fallo. |
| Command | Intención de cambiar estado, con datos suficientes para la operación. |
| Query y DTO | Solicitud de solo lectura y estructura de respuesta apropiada para la consulta. |
| Outbox | Registro técnico del evento en la misma transacción; su publicación externa requiere un consumidor posterior. |
| ADR | Registro de una decisión, su contexto, alternativas, consecuencias y trazabilidad. |
| SKU | Identificador único de un producto en el catálogo de FerreMax. |
| Producto | Herramienta o material que FerreMax cotiza, vende, transfiere o recibe en devolución. |
| Categoría | Agrupación comercial de productos; puede determinar políticas como devolución. |
| Existencia física | Cantidad contabilizada físicamente en una ubicación. |

| | |
|---|---|
| Cantidad comprometida | Cantidad asociada a reservas activas de pedidos confirmados. |
| Inventario disponible | Cantidad vendible calculada como existencia física menos comprometido, ajustada por movimientos confirmados. |
| Inventario en tránsito | Cantidad despachada en una transferencia que aún no fue recibida en destino. |
| Stock mínimo | Nivel configurado que genera una alerta de reabastecimiento. |
| Revalidación | Nueva comprobación de precio, disponibilidad y condiciones de una cotización vencida. |
| Crédito disponible | Monto que puede utilizar el cliente sin exceder su límite, sujeto a bloqueos y condiciones. |
| Excepción de crédito | Decisión documentada de Caja/Crédito fuera de la autorización normal. |
| Bitácora de auditoría | Historial no editable por usuarios operativos que registra acciones y cambios sensibles. |

A partir de este glosario se definen las entidades, value objects y aggregates con sus respectivas fichas técnicas:

**Entidades:**

Los términos siguientes que ya aparecen en el glosario ClienteEmpresarial, Producto, Ubicacion, PartidaPedido y Reserva, son las entidades de este dominio.

Una partida se identifica dentro de su pedido y puede conservar cantidades solicitadas, surtidas y pendientes. El código mínimo usa un registro Partida inmutable con número local, SKU, cantidad y precio; los estados por partida del proceso completo quedan modelados y pendientes de implementación.

**Value Objects:**

SKU, Dinero, Vigencia y, en el diseño general, DireccionEntrega y evidencia estructurada del resultado. No tienen una identidad de negocio independiente de sus valores. Dinero conserva importe y moneda; Vigencia conserva emisión y vencimiento.

**Fichas técnicas de Aggregates:**

1. Aggregate “Cotización”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | Cotizacion; partidas y Vigencia |
| Responsabilidades | Revalidar, emitir versiones y validar conversión. |
| Invariantes | Tiene folio, versión, vigencia y estado; no reserva; vencida requiere revalidación. |
| Eventos posibles | CotizacionRevalidada; conversión origina PedidoCreado. |

2. Aggregate “Pedido”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | Pedido; PartidaPedido y referencias de cotización |
| Responsabilidades | Confirmar, cancelar, conservar procedencia y aceptación. |
| Invariantes | Reserva ligada a pedido confirmado; cancelar antes del surtido libera; parcial requiere aceptación. |
| Eventos posibles | PedidoCreado, PedidoConfirmado, PedidoCancelado. |

3. Aggregate “InventarioUbicación”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | InventarioUbicacion; reservas por pedido |
| Responsabilidades | Reservar, liberar y calcular disponible por SKU y ubicación. |
| Invariantes | Disponible = física − comprometida; no negativo; tránsito no es stock utilizable en destino antes de recibir. |
| Eventos posibles | ReservaRegistrada/ReservaLiberada como candidatos; prototipo usa eventos de Pedido que describen la operación conjunta. |

4. Aggregate “CuentaCredito”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | CuentaCredito; condiciones y decisión excepcional |
| Responsabilidades | Evaluar condición normal y registrar excepción autorizada. |
| Invariantes | Cliente empresarial activo, sin bloqueo y con crédito suficiente; excepción requiere responsable y motivo. |
| Eventos posibles | CreditoEvaluado, ExcepcionCreditoResuelta. |

5. Aggregate “Transferencia”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | Transferencia; partidas y estados |
| Responsabilidades | Solicitar, autorizar/rechazar, despachar y recibir. |
| Invariantes | Origen, destino, responsable y estado obligatorios; recepción actualiza tránsito y destino con trazabilidad. |
| Eventos posibles | TransferenciaSolicitada, Despachada, Recibida o ConIncidencia. |

6. Aggregate “Entrega”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | Entrega; dirección, asignación y evidencia |
| Responsabilidades | Programar, registrar resultado e incidencia. |
| Invariantes | Salida requiere datos completos; cierre conserva receptor, fecha/hora y evidencia; incidencia no equivale a entregado. |
| Eventos posibles | EntregaProgramada, EntregaCompletada, IncidenciaEntregaRegistrada. |

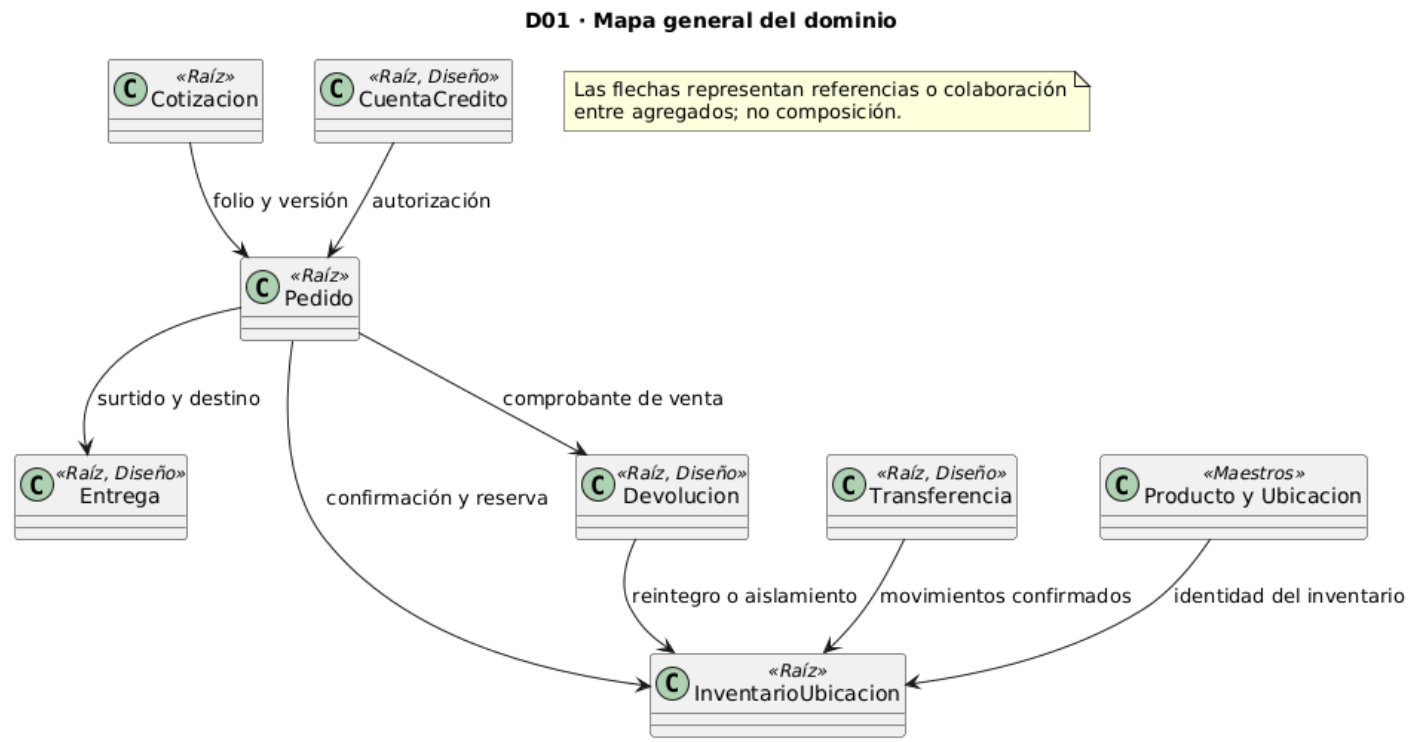
7. Aggregate “Devolucion”

| Elemento | Descripción |
|---|---|
| Raíz y contenido | Devolucion; partidas y evaluación |
| Responsabilidades | Evaluar elegibilidad y registrar destino financiero y físico. |
| Invariantes | Comprobante, categoría, condición y política configurable; no inventar plazos o porcentajes. |
| Eventos posibles | DevolucionAutorizada o Rechazada. |

**Domain Services:**

PoliticaCredito para evaluación normal y PoliticaSurtidoMultiorigen para priorizar ubicación local, almacén central y otras sucursales según BR07. PoliticaDevolucion evalúa categoría y condición cuando FerreMax defina sus parámetros. La última y la de surtido son propuestas de diseño, no clases ya ejecutables.

**Mapa General del Dominio:**




**Explicaciones dudosas o descartadas**:

1. Un único agregado para todo el proceso: Se descarta concentrar cotizaciones, pedidos, inventario y crédito en una sola raíz, porque tienen responsabilidades y ciclos de vida diferentes. Se mantienen agregados separados, relacionados mediante identificadores; esto requiere coordinar las operaciones que involucran varios de ellos.

2. Reservar inventario desde la cotización: Se descarta porque una cotización todavía no representa una compra confirmada. Cotizacion controla versiones y vigencia; la reserva corresponde a InventarioUbicacion y se vincula con un Pedido confirmado, conforme a BR01 y BR05.

3. Guardar la confirmación y la reserva en operaciones independientes: Se descarta porque un fallo podría dejar un pedido confirmado sin inventario reservado. Pedido e InventarioUbicacion conservan sus límites de dominio, pero sus cambios se confirman o revierten juntos mediante Unit of Work, según ADR003 (más adelante en el informe).

4. Representar el inventario como una cantidad global por producto: Se descarta porque ocultaría dónde se encuentran las existencias y dificultaría aplicar la prioridad de surtido.

InventarioUbicacion controla cantidades y reservas por SKU y ubicación; la disponibilidad global se obtiene mediante una consulta que reúne esa información, conforme a BR04 y BR07.

## Etapa 4: Service Layer

Mediante la Service Layer se diseñan los servicios de aplicaciones que exponen los casos de uso:

| Servicio | Método | Entrada | Colaboradores | Salida | Reglas delegadas |
|---|---|---|---|---|---|
| CotizacionService | revalidar | Actor, ID, versión, vigencia, partidas, condiciones y motivo | UnitOfWork; CotizacionRepository; InventarioRepository; Cotizacion | Nueva versión y folio | BR01-03; validación de partidas y vigencia. |
| CotizacionService | convertir | Actor, ID, versión y aceptación | CotizacionRepository; InventarioRepository; PedidoRepository; Cotizacion | ID, estado, folio/versión y disponibilidad consultada | Vigencia y aceptación en Cotizacion; no reserva. |
| PedidoService | confirmar | Actor Caja, pedido, AutorizacionPago y ubicación | PedidoRepository; InventarioRepository; UnitOfWork; Pedido; InventarioUbicacion | ID, estado, referencia y ubicación | Pago y transición en Pedido; disponibilidad y reserva en InventarioUbicacion. |
| PedidoService | cancelar | Actor con permiso, pedido y motivo | PedidoRepository; InventarioRepository; UnitOfWork; Pedido | ID y estado | Estado y surtido en Pedido; liberación en InventarioUbicacion. |
| Handlers de consulta | handle | Actor y SKU o ID de pedido | DisponibilidadReader o PedidoReader | DTO independiente | Autorización de acceso; consultas de solo lectura. |

**Revisión obligatoria:**

• ¿El nombre del método expresa una acción del negocio?

Sí, el método expresa una acción lógica dentro del negocio, desde confirmar un pedido hasta revalidarlo.

• ¿El método coordina o contiene decisiones que deberían estar en el dominio?

Los métodos contienen decisiones en el dominio, como revalidar o cancelar un pedido, acciones que continúan o interrumpen el flujo del negocio.

• ¿El servicio depende de interfaces o de detalles concretos de base de datos?

No, dependen de consultas a la base de datos, pero no de detalles específicos como la estructura de la misma.

• ¿La operación tiene un inicio y un final transaccional comprensible?

Sí, se define con las entradas y salidas presentes en la tabla, con datos de entrada y resultados esperados de cada servicio.

• ¿Los errores del dominio pueden explicarse con términos del negocio?

Si hay un error en el dominio es debido a un fallo en el negocio, como reserva sin disponibilidad, por lo que si pueden explicarse en términos del negocio.

## Etapa 5: Separar persistencia y transacciones

Se definen las interfaces de repository mediante la siguiente tabla, y después el Unit of Work (UoW):

| Repository | Necesidad del dominio | Métodos propuestos | Objeto que recupera o guarda |
|---|---|---|---|
| **CotizacionRepository** | Recuperar una versión específica y conservar una nueva sin sobrescribir las anteriores. | obtener(cotizacion_id, version); guardar_nueva_version(cotizacion) | Cotizacion, con sus partidas, condiciones y vigencia. |
| **PedidoRepository** | Recuperar un pedido y guardar sus cambios de estado. | obtener(pedido_id); guardar(pedido) | Pedido, con sus partidas y referencia a la cotización de origen. |
| **InventarioRepository** | Recuperar inventario para modificarlo, conservar sus reservas e identificar la ubicación reservada para un pedido. | obtener_para_actualizar(sku, ubicacion_id); guardar(inventario); ubicacion_reserva(pedido_id) | InventarioUbicacion y sus reservas; el último método devuelve el identificador de ubicación. |

| | | | |
|---|---|---|---|
| **DisponibilidadReader** | Consultar la disponibilidad global de productos sin materializar guardados ni hacer cambios. | consultar_disponibilidad(sku,ubicación_id) | Información mediante consultas parametrizadas devolviendo cuatro cantidades de disponibilidad, una por ubicación. |
| **PedidoReader** | Hacer un resumen de un pedido para la lectura del usuario mediante consultas sin cambios. | consultar_pedido(pedido_id, cotización_id) | Información y proyección del pedido, estado, cotización, total, etc. |

Las consultas informativas se atienden mediante DisponibilidadReader y PedidoReader, que devuelven modelos de lectura y no permiten guardar cambios. En PostgreSQL, los repositorios de escritura comparten la conexión de la unidad de trabajo.

**Unit of Work:**

UnitOfWork coordina los repositorios de cotizaciones, pedidos e inventario para confirmar juntos los cambios de cada operación. También registra la auditoría y el evento correspondiente en la outbox. El contrato contempla abrir y cerrar la unidad de trabajo, registrar auditoría y eventos, y confirmar mediante "commit("`. Si se abandona la operación sin confirmar, PostgreSQL ejecuta rollback; Memory descarta los cambios de su copia de trabajo.

## Etapa 6: Separar commands y queries

Se utiliza el CQRS básico para diferencia el cambio de estado de las consultas, utilizando dos operaciones de commands y dos operaciones de queries:

| Tipo | Operación | Handler | Dependencias | Resultado |
|---|---|---|---|---|
| Command | ConfirmarPedido | ConfirmarPedidoHandler | PedidoService; UoW; repositorios de pedido e inventario | Confirmación y reserva, o error de negocio. |
| Command | CancelarPedido | CancelarPedidoHandler | PedidoService; UoW; repositorios de pedido e inventario | Cancelación y liberación, o rechazo. |
| Query | ConsultarDisponibilidad | ConsultarDisponibilidadHandler | DisponibilidadReader | SKU, encontrado y lista de ubicaciones con cantidades. |
| Query | ConsultarPedido | ConsultarPedidoHandler | PedidoReader | ID, estado, folio/versión de cotización, total y moneda. |

Se conserva una sola base PostgreSQL. Los DTO son estructuras de lectura y no se usan para modificar automáticamente entidades. Los comandos delegan al servicio para conservar un único recorrido de reglas y transacción; no duplican la lógica en el handler.

La emisión y revalidación pueden exponerse inicialmente como métodos del servicio. No es necesario añadir handlers idénticos solo para incrementar clases. Así, la separación obligatoria queda demostrada con los dos comandos y las dos consultas seleccionadas.

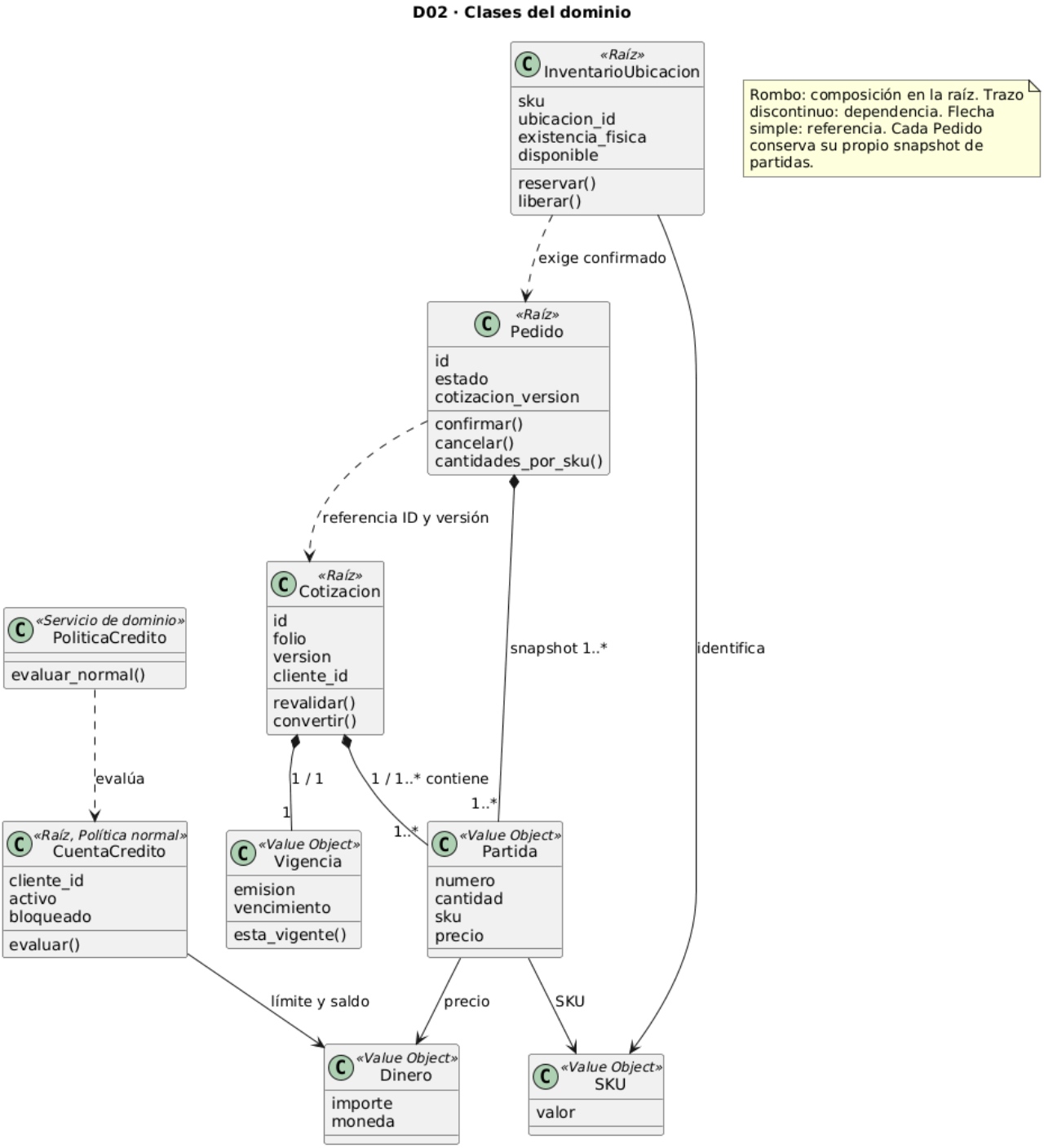
## Etapa 7: Revisión con principios SOLID

Antes de pasar a los diagramas, debe revisarse la propuesta con los principios SOLID con hallazgos en la propuesta arquitectural de FerreMax:

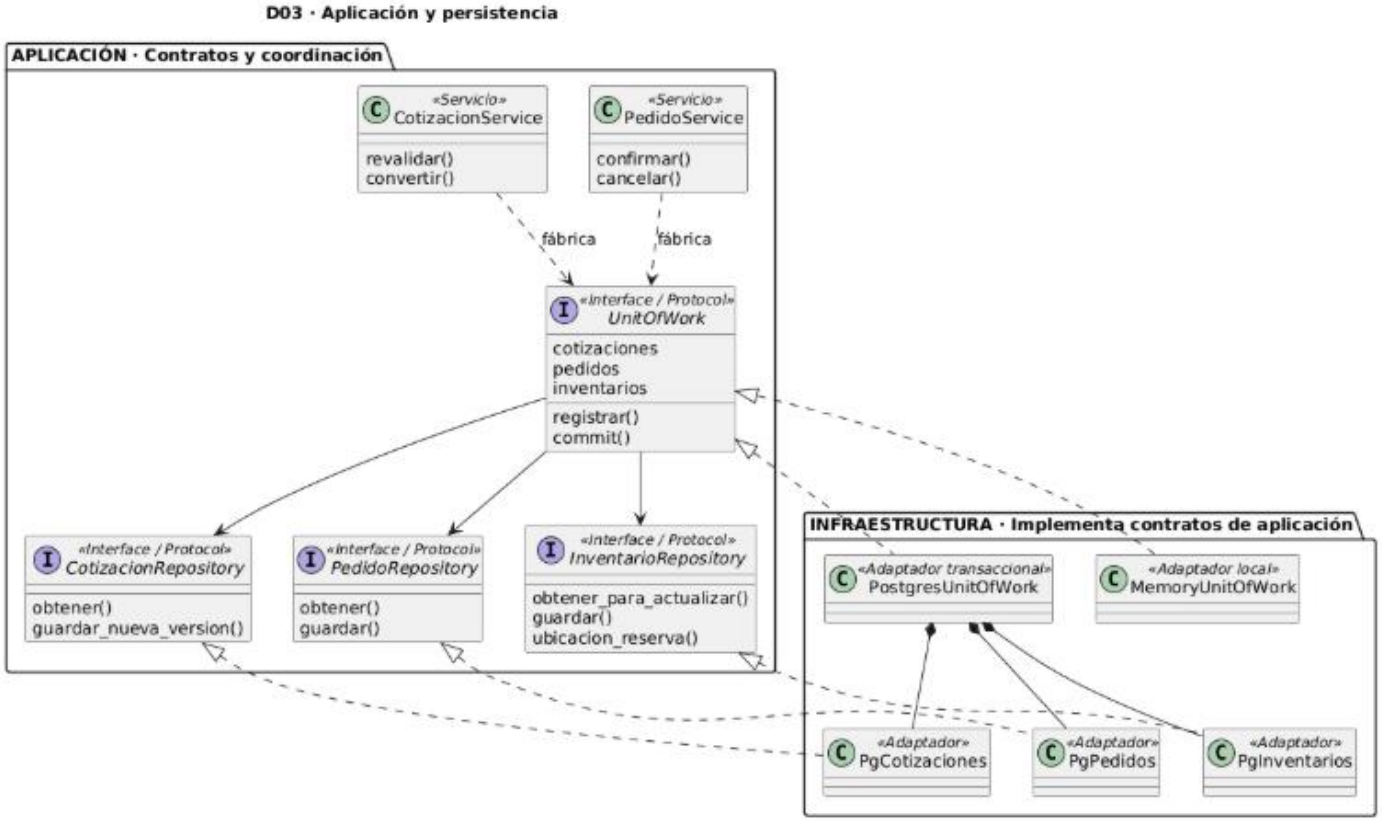
| Principio | Evidencia | Impacto | Cambio o control |
|---|---|---|---|
| S (Single Resp) | Las reglas de vigencia y reserva están en Cotizacion e InventarioUbicacion; el servicio coordina. | Mezclarlas con SQL haría cambiar el servicio por políticas y por almacenamiento. | Separar model.py, services.py y postgres.py; mover ubicacion_reserva al repositorio de inventario. |
| O (Open / Closed) | PoliticaCredito expresa evaluación normal; devolución y surtido tienen variaciones pendientes. | Una cadena creciente de condiciones mezclaría políticas distintas. | Mantener políticas separadas; añadir implementaciones solo cuando haya variantes justificadas, sin inventar categorías. |
| L (Liskov subst) | Repositorios Memory y PostgreSQL ofrecen operaciones equivalentes y rechazos de negocio. | Una implementación que pierde versiones o permite reservas duplicadas rompería el contrato. | Conservar versiones, rechazar SKU inexistente y aplicar rollback. Prueba de contrato PostgreSQL pendiente de conexión. |
| I (Interface segr) | DisponibilidadReader y PedidoReader no obligan a una consulta a guardar o eliminar. | Una interfaz universal expondría métodos irrelevantes al lado lector. | Contratos pequeños para consultas y repositorios especializados por agregado. |
| D (Dep inversión) | Servicios reciben una fábrica UnitOfWork y un reloj; el dominio no importa SQL ni Psycopg. | Dependencia directa del backend impediría revisar el dominio sin una conexión. | Adaptadores PostgreSQL y Memory; consultas parametrizadas y composición externa de dependencias. |

## Etapa 8: Representar y documentar la arquitectura

**Diagrama de clases**:

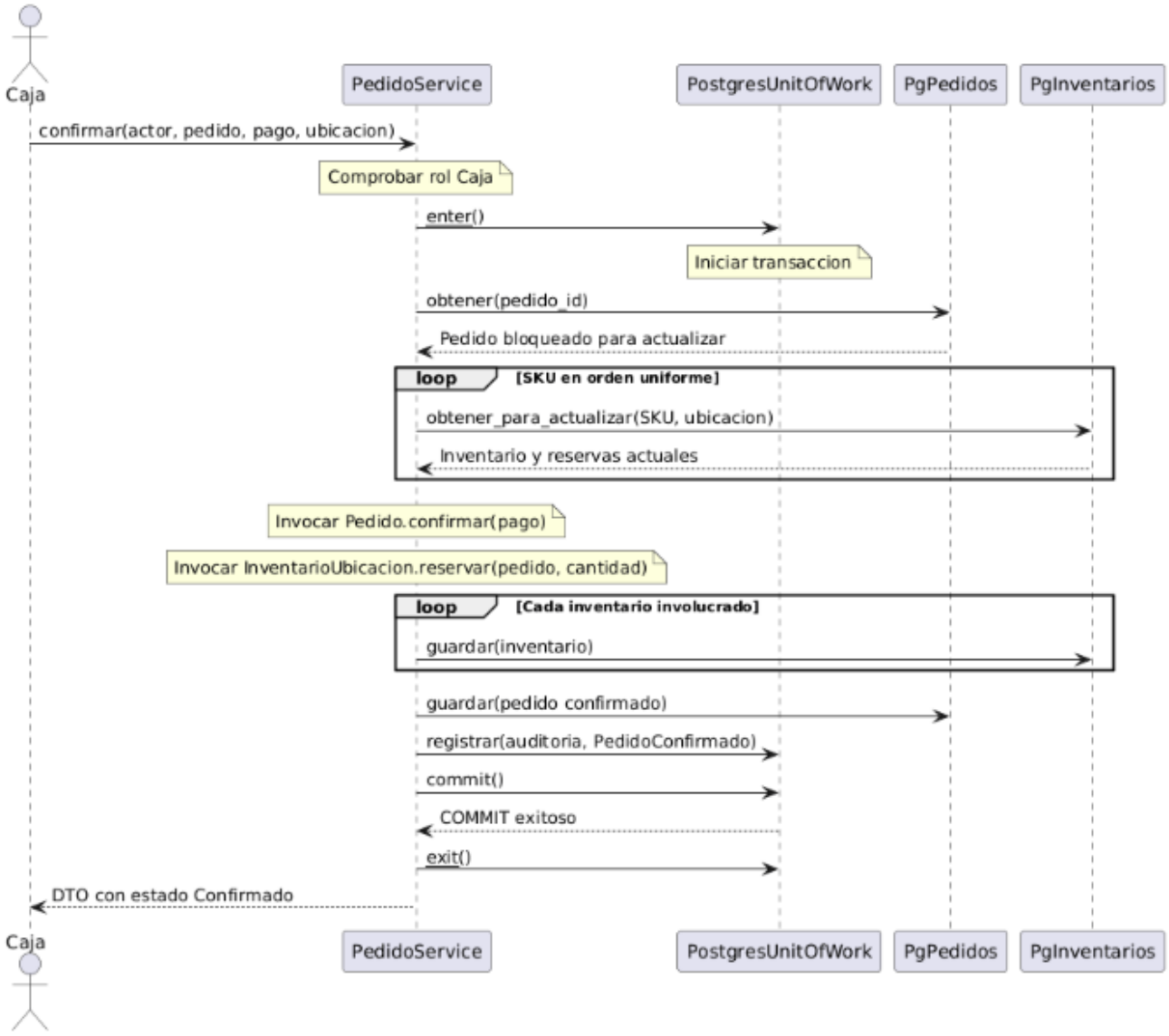


## Diagrama de aplicación e infraestructura:

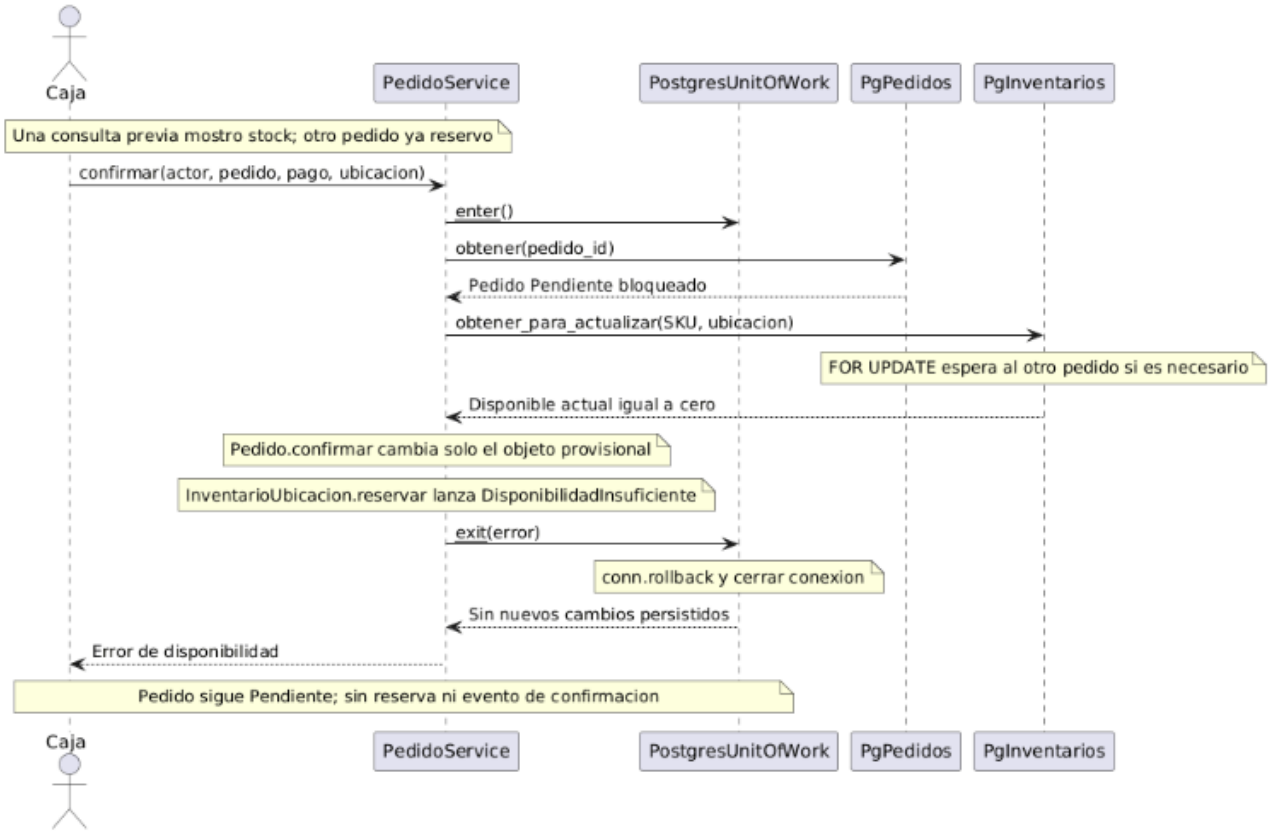


Después se formaron diagramas de secuencia, de un flujo principal y un flujo interrumpido por una regla o excepción:

**Diagrama de flujo principal: Confirmación y reserva:**



## Diagrama de secuencia de flujo interrumpido por unidad ya comprometida:



Después de estos diagramas, se registran las decisiones arquitecturales mediante registros de decisión arquitecturales (ADR's):

## ADR-001: Domain Model como patrón principal

| Campo | Contenido |
|---|---|
| **Estado** | Propuesta para revisión del equipo. |
| **Contexto** | Las cotizaciones conservan versiones y vigencia; el pedido cambia de estado; inventario y reservas deben mantenerse consistentes. Esas reglas reaparecen en varios casos de uso. |
| **Alternativas** | Transaction Script ofrece menor estructura inicial, con riesgo de duplicar reglas. Table Module organiza conjuntos de datos, pero coloca el ciclo de vida del pedido fuera de los objetos que representan el negocio. Domain Model encapsula reglas y conserva identidad y comportamiento. |
| **Decisión** | Proponer Domain Model en la escritura y consultas SQL con DTO propios en la lectura. No clasificar cualquier SELECT como Table Module ni introducir una combinación de patrones sin delimitarla. |
| **Consecuencias** | Beneficio: responsabilidades visibles y reglas reutilizables. Costo: más objetos, contratos y validación de límites. Riesgo: clases anémicas o servicios que absorban el dominio. Control: métodos de negocio y cinco hallazgos SOLID. |
| **Trazabilidad** | BN02/BN03; BR01–06; RNF02; US04/US05/US09/US10; CU02–CU05. |

## ADR002 Python y PostgreSQL alojado en Supabase

| Campo | Contenido |
|---|---|
| **Estado** | Tecnología y organización confirmadas por el equipo; proyecto aún no creado. |
| **Contexto** | Se propuso PostgreSQL en Supabase y se autorizó continuar con esa opción; se eligió Santana's Org como repositorio en Github para un proyecto nuevo. |
| **Alternativas** | SQLite simplifica una demostración local. PostgreSQL local conserva las funciones del motor pero requiere instalación/operación. Supabase aloja PostgreSQL y permite administración y persistencia compartida. |
| **Decisión** | Python organiza el dominio y la aplicación. Un adaptador PostgreSQL conserva transacciones y repositorios. Supabase se utiliza inicialmente para alojar la base; se conserva una única base de lectura y escritura. |
| **Consecuencias** | Beneficio: práctica con persistencia compartida y transacciones de PostgreSQL. Costo: conexión, permisos, dependencias y aprovisionamiento. El costo de creación debe verificarse antes de activar un proyecto; no se asume que una organización sea gratuita. No hay proyecto activo de FerreMax ni pruebas conectadas reportadas. |
| **Trazabilidad** | RNF02/03/04/08; BR04/05/16; CU04/CU05; conversación de decisión tecnológica y Release. |

## ADR003 Unit of Work para confirmación y reserva

| Campo | Contenido |
|---|---|
| **Estado** | Propuesta implementada y revisada con Memory; validación PostgreSQL pendiente. |
| **Contexto** | AC-US09-01 exige reserva atómica y AC-US09-02 impide asignar la última unidad a dos pedidos. Guardar el estado del pedido separado de la reserva puede producir confirmaciones incoherentes. |
| **Alternativas** | Guardar por separado introduce cambios parciales. Coordinar con consistencia eventual añade recuperación y estados de compensación aún no definidos. Una transacción local puede proteger los cambios requeridos en una misma base. |
| **Decisión** | El servicio abre UoW, bloquea pedido e inventarios en orden, invoca al dominio, guarda pedido/reservas/auditoría/outbox y confirma una sola transacción. Ante error, revierte la operación completa. |
| **Consecuencias** | Beneficio: confirmación y compromiso de inventario quedan juntos. Costo: retención breve de bloqueos y manejo de contención. La transacción abarca más de un agregado sin convertirlos en una sola raíz. Las conexiones y los bloqueos reales necesitan ensayo PostgreSQL; RNF01 no se declara cumplido sin medirlo. |
| **Trazabilidad** | BN02; BR04-06/BR16; RNF02/RNF04; AC-US06-01/02; AC-US09-01/02; CU04/CU05. |

## ADR004 CQRS básico sobre una sola base

| Campo | Contenido |
|---|---|
| **Estado** | Propuesta implementada en comandos y consultas seleccionados. |
| **Contexto** | La consulta global debe distinguir física, comprometida, tránsito y disponible. Confirmar y cancelar cambian estado y exigen validaciones diferentes a una consulta. |
| **Alternativas** | Un modelo único para pantallas y escritura simplifica estructuras, pero puede permitir modificaciones involuntarias. Bases separadas exigirían sincronización y operación adicional. Separar handlers y modelos conservando PostgreSQL resuelve la necesidad actual. |
| **Decisión** | ConfirmarPedido y CancelarPedido son Commands. ConsultarDisponibilidad y ConsultarPedido son Queries con contratos de lectura y DTO propios. Los commands delegan a la Service Layer; una consulta no recibe métodos para guardar. |
| **Consecuencias** | Beneficio: intención explícita y lectores pequeños. Costo: DTO y handlers adicionales. No implica Event Sourcing, réplicas ni dos bases. La separación se aplica donde aporta claridad y no obliga a crear un handler decorativo por cada método. |
| **Trazabilidad** | BN01/BN02; BR04-06; RNF01/RNF06; US01/US02/US09/US10; CU01/CU04/CU05. |

## ADR005 Auditoría y outbox junto con el cambio de negocio

| Campo | Contenido |
|---|---|
| **Estado** | Propuesta implementada como registro; consumidor externo pendiente. |
| **Contexto** | BR16 y RNF04 exigen quién, cuándo, qué cambió y por qué. Publicar un evento antes del COMMIT puede comunicar una confirmación que posteriormente falla. |
| **Alternativas** | Auditar/publicar por separado puede perder registros o emitir hechos no confirmados. Añadir un intermediario de mensajes externo aumenta trabajo sin un consumidor definido. Guardar auditoría y evento junto con el cambio permite conservar evidencia coherente. |
| **Decisión** | Registrar auditoría y evento en la misma transacción. El backend operativo tiene permiso de insertar y leer auditoría, sin actualizarla ni borrarla. Una outbox técnica conserva eventos pendientes para un consumidor posterior. |
| **Consecuencias** | Beneficio: coherencia entre cambio y evidencia. Costo: tabla adicional y necesidad futura de publicación/idempotencia. El prototipo registra eventos, pero no afirma entregar notificaciones ni garantizar exactamente una entrega externa. |
| **Trazabilidad** | BN10; BR16; RNF04; CA-US20-01; CU02-CU05. |

## Bitácora de IA:

Para la realización de este diseño arquitectural se utilizó la IA con resultados supervisados por el equipo y correcciones para congruencia del diseño. A continuación, se muestra la bitácora de uso de la IA:

| Itera | Objetivo | Prompt | Respuesta útil | Error o supuesto detectado | Decisión del equipo |
|---|---|---|---|---|---|
| IA01 | Decidir el patrón de lógica de negocio a partir de las reglas de negocio y los cinco casos de uso. | "Analiza la siguiente información. Se trata de reglas de negocio, y cinco casos de uso para el diseño arquitectural de un sistema. Compara Transaction Script, Table Module y Domain Model, indica las ventajas y desventajas y en base a eso y posibles consecuencias, indica cuál consideras el patrón ideal para utilizar. | Resumen de los tres patrones de lógica de negocio, detalles de ventajas, desventajas y consecuencias, y un análisis profundo del negocio en cada patrón. Al final, IA consideró Domain Model como patrón ideal. | Se encontraron OB01/CE01 incongruentes, además de pequeños errores en las descripciones de los patrones, pero en términos generales se desarrolló correctamente. | Se analizó el resumen de IA y se comparó con la idea de lo visto en clase, se corroboró la idea del equipo y se decidió por Domain Model como patrón a usar. |

| | | | | | |
|---|---|---|---|---|---|
| IA02 | Construir el dominio del sistema | "Según el patrón de lógica escogido, analiza este glosario técnico [glosario corregido de la unidad 3] y las reglas de negocio, propón candidatos a entidades, value objects, aggregates y domain events. Incluye la evidencia para cada candidato y las dudas o incongruencias que encuentres para validación humana" | Lista de varios candidatos para cada elemento del dominio con sus respectivas evidencias de reglas y necesidades, así como las dudas que necesitan validación del equipo. | Algunos candidatos se escapaban de las reglas y el glosario del negocio, por lo que se descartaron. También hubo dudas que eran redundantes e ilógicas. | Se seleccionaron candidatos que no se escaparan de las reglas del negocio y que concordaran con el flujo establecido en la unidad 2. Se seleccionaron solo las dudas que fueran realmente relevantes. |
| IA03 | Construir la Service Layer | "Se eligieron los siguientes candidatos para los elementos del dominio [elementos elegidos de la interacción anterior]. A partir de estos y de las reglas anteriores, propón Application Services." | Entregó una tabla con varios servicios, enlazados con sus reglas, entradas, salidas y colaboradores. | Algunos servicios no estaban respaldados por suficientes reglas y otros no tenían congruencia con las mismas. | Hubo servicios suficientes para elegirlos para la tabla de service layer, y se corroboraron los enlaces con sus elementos. |
| IA04 | Encontrar incongruencias SOLID y validar la arquitectura diseñada. | Analiza el siguiente diseño arquitectural [dominio, service layer y reglas de negocio, UoW, etc.] y busca violaciones a SOLID. No reescribas información ni modifiques lo adjunto, sólo señala las violaciones, identifica el riesgo con evidencia y propón alternativas para solucionarlas" | Encontró una cantidad de violaciones SOLID e identificó a cuales elementos hacía referencia. Dio evidencia y propuso alternativas para solucionarlo. | Una violación era incongruente por lo que se descartó. El resto era correcta y solucionables de manera sencilla. | Se seleccionaron aquellas violaciones que requerían cambios mayores para incluirlas. Aquellas que no, se solucionaron fácilmente y se omitieron. |
| IA05 | Diseñar diagramas con IA para revisarlos y pasarlos a PlantUML. | "En base al dominio del negocio [Se adjunta información del dominio] construye un diagrama de clases. No agregues ninguna clase | Un diseño inicial de diagrama de clase del dominio utilizado en el negocio. | Agregó una relación adicional entre dos clases no relacionadas. El resto fue un | Se le indicó a la IA que eliminara la relación sobrante y se construyó otro |

| | | | | | |
|---|---|---|---|---|---|
| | | adicional ni ningún elemento que no esté en esta información, y mantenla tal como está." | | diagrama construido correctamente. | diagrama correctamente. |
| IA06 | Realizar diagramas de secuencia iniciales para el flujo completo y flujo interrumpido. | "A partir del siguiente BPMN del negocio, [BPMN de la unidad 2] y las reglas del negocio, además del dominio construido, construye un diagrama de secuencia de un proceso completo y un proceso interrumpido por falta de disponibilidad. No incluyas actores ni procesos no incluidos en el dominio" | Un diagrama de secuencia de un flujo completo de inicio a fin, y un diagrama de secuencia interrumpido por falta de disponibilidad. | Los diagramas eran simples pero correctos, sólo que no indicaban explícitamente los actores. | Se indicó a la IA agregar los actores sin modificar lo demás, y se entregaron diagramas correctos. |
| IA07 | Cambiar los diagramas de IA a código para PlantUML | "Utiliza este diagrama [diagrama realizado y corregido por IA] y conviertelo a código para realizarlo en PlantUML. No cambies ninguna información que aparezca en el diagrama" (Se usó este prompt para los diagramas realizados por IA) | Código para realizar los mismos diagramas en PlantUML. | El código era correcto para realizarlo en PlantUML. | Se utilizó el código en PlantUML y se verificó que no se perdiera información. |

## Etapa 9. Validación de la propuesta

| Control | Pregunta | Resultado |
|---|---|---|
| Trazabilidad | ¿Cada componente importante responde a una necesidad, regla o caso de uso? | Todos los componentes y elementos del diseño están relacionados con BN, BR, US y/o escenarios existentes. |
| Lenguaje | ¿Los mismos términos significan lo mismo en BRD, UML, código y ADR? | Sí, en el caso de incongruencias se anotaron y se aplicaron soluciones sin modificar los términos anteriores. Términos del glosario conservados; vocabulario técnico añadido con definición. |

| | | |
|---|---|---|
| Responsabilidad | ¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación? | Las reglas son la base en el dominio, coordinación en servicios, e incluso SQL en infraestructura. |
| Consistencia | ¿Los diagramas de clase y secuencia representan la misma arquitectura? | Se construyeron los diagramas a partir de la arquitectura y el dominio previamente construido, se verificó todo. |
| Persistencia PostgreSQL | ¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura? | Sí, se construyó el dominio a partir del patrón escogido y con una estructura no dependiente en su totalidad de pequeños detalles. |
| Transacción | ¿El límite de la operación y su manejo de error están definidos? | Según las reglas y el domino creado se definieron los límites del dominio. |
| IA | ¿Cada aporte de IA fue revisado y su uso esta registrado? | Iteraciones y correcciones registradas con revisión humana. |

### Matriz de trazabilidad por necesidad:

| ID | Reglas y RNF | Historias | Caso o alcance | Componentes | ADR | Cobertura |
|---|---|---|---|---|---|---|
| BN01 | BR04; RNF01 | US01/US02 | CU01 | ConsultarDisponibilidadHandler; DisponibilidadReader | ADR004 | Consulta implementada; objetivo temporal sin medir. |
| BN02 | BR04-06; RNF02 | US09/US10 | CU04/CU05 | Pedido; InventarioUbicacion; UnitOfWork | ADR001/003 | Reserva y liberación de contado en una ubicación. |
| BN03 | BR01-03; RNF08 | US03-05 | CU02/CU03 | Cotizacion; CotizacionService; CotizacionRepository | ADR001 | Revalidación y conversión implementadas; emisión inicial simulada. |
| BN04 | BR07/08 | US05/US11/US14/US15 | CU04 más diseño general | Pedido; PoliticaSurtidoMultiorigen; Transferencia | ADR001/003 | Multiorigen y surtido diseñados; prototipo local. |
| BN05 | BR09-11 | US07/US08 | Diseño comercial | CuentaCredito; PoliticaCredito | ADR001 | Evaluación normal probada; excepciones y persistencia pendientes. |
| BN06 | BR12/13 | US11-13 | Diseño logístico | Transferencia; InventarioUbicacion | ADR001/003 | Mapeado; sin flujo ejecutable de transferencia. |

| | | | | | | |
|---|---|---|---|---|---|---|
| BN07 | BR08 | US14 | Diseño comercial | Pedido; PartidaPedido; aceptación del cliente | ADR001 | Mapeado; sin aplicación ejecutable de parciales. |
| BN08 | BR14; RNF07 | US16/US17 | Diseño logístico | Entrega; DireccionEntrega; EvidenciaEntrega | ADR001 | Mapeado; sin programación de unidad ni captura operativa. |
| BN09 | BR15; RNF08 | US18 | Diseño de postventa | Devolucion; PoliticaDevolucion; Categoria | ADR001 | Política exacta pendiente del negocio. |
| BN10 | BR16; RNF04 | US19/US20 | CU02-CU05 y diseño de panel | Auditoria; Outbox; DTO de reportes | ADR005 | Auditoría local implementada; panel completo pendiente. |

**Matriz de trazabilidad por regla:**

| Regla | Escenarios U2 | Caso o alcance | Componente responsable | Evidencia o límite |
|---|---|---|---|---|
| BR01 | US03/US04/US05 | CU02/CU03 | CotizacionService | No llama reservar durante revalidación/conversión; prueba sin stock comprometido. |
| BR02 | US03/US04 | CU02/CU03 | Cotizacion; CotizacionRepository | Folio, versión y vigencia conservados; estado vigente se deriva de Vigencia. |
| BR03 | AC-US04-01/02 | CU02/CU03 | Cotizacion.convertir/revalidar | Vencida rechazada; versión anterior conservada. |
| BR04 | US02; AC-US09-01 | CU01/CU04 | InventarioUbicacion.disponible | Física menos reservas; tránsito informado por separado. |
| BR05 | US06; AC-US09-01 | CU04 | Pedido.confirmar; InventarioUbicacion.reservar | Dominio exige confirmado; UoW guarda estado y reserva juntos. |
| BR06 | AC-US10-01 | CU05 | Pedido.cancelar; InventarioUbicacion.liberar | Cancelación antes de surtido libera; reducción es variante futura. |
| BR07 | US01/US11 | Diseño general | PoliticaSurtidoMultiorigen | Prioridad local, almacén, otras sucursales; prototipo no implementa esta selección. |

| | | | | |
|---|---|---|---|---|
| BR08 | AC-US14-01/02 | Diseño general | Pedido y PartidaPedido | Aceptación explícita para parciales; no usar excepción de stock como consentimiento. |
| BR09 | AC-US07-01 | Diseño general | CuentaCredito.evaluar | Activo es condición de autorización normal. |
| BR10 | AC-US07-01/02 | Diseño general | PoliticaCredito | Bloqueo o insuficiencia devuelven EnRevision. |
| BR11 | AC-US08-01/02 | Diseño general | CuentaCredito y decisión excepcional | Motivo, responsable, fecha y condición requeridos; no inventar umbrales de excepción. |
| BR12 | AC-US11-01; US12 | Diseño logístico | Transferencia | Origen, destino, partidas, responsable y estado. |
| BR13 | AC-US13-01/02 | Diseño logístico | Transferencia e InventarioUbicacion | Recepción actualiza tránsito/destino; diferencia se conserva como incidencia. |
| BR14 | AC-US17-01/02 | Diseño logístico | Entrega | Evidencia de receptor y resultado; incidencia no cierra como entregado. |
| BR15 | CA-US18-01 | Diseño postventa | Devolucion y PoliticaDevolucion | Evaluación por categoría/condición y política pendiente. |
| BR16 | CA-US20-01 | CU02-CU05 | UnitOfWork.registrar; auditoria | Valores anteriores/nuevos, actor, fecha y motivo dentro de la transacción. |

### Trazabilidad de los rechazos:

| Escenario | Entrada o situación | Resultado y comprobación |
|---|---|---|
| AC-US04-01 | Cotización fuera de vigencia | CotizacionVencida; no crea pedido. |
| AC-US06-02 | Pago rechazado | Pedido sigue Pendiente; sin reserva ni evento de confirmación. |
| AC-US09-02 | Dos pedidos intentan la última unidad | Uno confirma; otro recibe DisponibilidadInsuficiente. Memory probado; PostgreSQL pendiente. |
| AC-US10-01 | Cancelación autorizada antes del surtido | Estado Cancelado y stock disponible restituido. |
| AC-US01-02 propuesto | SKU inexistente | Encontrado falso y lista vacía; ID original repetido registrado en H01. |

## Conclusiones

La arquitectura conecta las necesidades de FerreMax con componentes concretos: Cotizacion conserva vigencia y versiones; Pedido expresa confirmación y cancelación; InventarioUbicacion protege la disponibilidad. La Service Layer coordina esos componentes y el Unit of Work evita que una confirmación quede guardada sin las reservas correspondientes.

Domain Model se propone por la interacción entre reglas y estados, y acepta como costo una estructura mayor que Transaction Script. CQRS se mantiene básico, con una sola base y modelos de lectura propios. PostgreSQL en Supabase permite continuar el proyecto y revisar persistencia compartida, aunque añade configuración y requiere validar sus transacciones de forma real.

La prueba del diseño incluye también los rechazos. La cotización vencida, el pago rechazado y la disponibilidad insuficiente deben tener resultados comprensibles y conservar un estado coherente. Los procesos no implementados y las políticas pendientes se identifican expresamente para mantener la continuidad con U2.

El uso de IA es una herramienta muy útil siempre y cuando sea usada con prudencia y con validación humana, para que ayude a construir algo coherente y sin errores graves.